

Reviewer Pack — Real Algebraic Geometry of XOR Circuit Weights Bounds AC^0 Pa...

Entry 67d9d6a3fd41 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 15:00:33 UTC

Real Algebraic Geometry of XOR Circuit Weights Bounds AC^0 Parity Depth

Entry ID: 67d9d6a3fd41 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 15:00:33 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For any AC^0 circuit C computing the PARITY function on n inputs, there exists a real algebraic variety V associated with C such that the number of irreducible components of V is at least $c \cdot \log(\text{size}(C))$ for some absolute constant c .’, ‘If C computes XOR instead of PARITY, then the number of irreducible components of the associated variety V is less than $c \cdot \log(\text{size}(C))$.’, ‘For all other AC^0 circuits computing a different function f , the number of irreducible components of the associated variety V is bounded by some function $g(n)$ independent of the size of C .’]

Rationale (proposer’s reasoning):

[‘Real algebraic geometry provides a powerful tool for analyzing the structure of Boolean functions.’, ‘The irreducibility of algebraic varieties corresponds to non-trivial properties of the underlying Boolean function.’, ‘If this conjecture is true, it would provide a new way to understand the complexity of PARITY and XOR circuits.’]

Taxonomy category: AC^0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ce443755edea4e9e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The number of irreducible components in the associated real algebraic variety V , for a given AC^0 circuit C computing XOR, is less than $c \cdot \log(\text{size}(C))$ where c is an absolute constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - real algebraic geometry AND XOR circuit complexity-AC⁰ parity depth IN real algebraic geometry-irreducible components variety V AND Boolean circuit complexity

Top relevant hits considered: - [<http://arxiv.org/abs/1409.1534v1>] Algorithms in Real Algebraic Geometry: A Survey - [<http://arxiv.org/abs/1209.3595v2>] Noncommutative complex differential geometry - [<http://arxiv.org/abs/math/0108225v1>] Amoebas of algebraic varieties - [<http://arxiv.org/abs/1912.00347v3>] Equiresidual algebraic geometry I: The affine theory - [<http://arxiv.org/abs/math/0107179v2>] Enumerative Real Algebraic Geometry - [<http://arxiv.org/abs/2508.18081>] GWTC-4.0: Methods for Identifying and Characterizing Gravitational-wave Transients - [<http://arxiv.org/abs/2503.23293v1>] The DUNE Phase II Detectors

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_primes(k):
        primes = []
        num = 2
        while len(primes) < k:
            if is_prime(num):
                primes.append(num)
            num += 1
        return primes
```

```

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x

def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0.0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    sign = 1
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += sign * A[0][j] * determinant(submatrix)
        sign *= -1
    return det

def generate_ac0_circuit(n, func):
    if n == 1:
        return [func]
    else:
        left = generate_ac0_circuit(n // 2, lambda x: x[0])
        right = generate_ac0_circuit(n // 2, lambda x: x[1])
        return [(left[i], right[i]) for i in range(len(left))]

def count_irreducible_components(circuit):
    if not circuit:
        return 0
    n = len(circuit)
    A = [[0.0] * (n + 1) for _ in range(n + 1)]

```

```

    b = [0.0] * (n + 1)
    for i in range(n):
        A[i][i] = 1.0
        b[i] = circuit[i]
    A[n][n] = 1.0
    x = gaussian_elimination(A, b)
    components = set()
    for i in range(n):
        if x[i] != 0:
            components.add(i)
    return len(components)

n_values = [5, 10, 15, 20, 30, 40]
total_components = 0
num_trials = 0

for n in n_values:
    circuit = generate_ac0_circuit(n, lambda x: x[0] ^ x[1])
    components = count_irreducible_components(circuit)
    total_components += components
    num_trials += 1

mean_components = total_components / num_trials
std_components = math.sqrt(sum((components - mean_components) ** 2 for components in range(total_components)))

conjecture_holds = mean_components < 0.5 * math.log(num_trials)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "irreducible_components",
    "metric_value": mean_components,
    "instances_tested": num_trials,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else generate_primes(30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_components = sum(result["metric_value"] for result in results) / len(results)
    std_components = math.sqrt(sum((result["metric_value"] - mean_components) ** 2 for result in results)) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_components} std={std_components} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_components} std={std_components} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])

```

```
print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ec0bcdcd.py", line 135, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ec0bcdcd.py", line 111, in run_trial
    components = count_irreducible_components(circuit)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ec0bcdcd.py", line 98, in count_irreducible_components
    x = gaussian_elimination(A, b)
       ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ec0bcdcd.py", line 51, in gaussian_elimination
    b[j] -= factor * b[i]
           ^^^^^^
```

TypeError: can't multiply sequence by non-int of type 'float'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's claim. | next: Investigate the cause of the crash and attempt to run the test again with appropriate error handling.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12700
2	propose	ollama_remote	glm4:latest	0	0	10631
3	propose	ollama_remote	glm4:latest	0	0	10280
4	preregistration	ollama_remote	glm4:latest	0	0	5792
5	novelty	ollama_remote	glm4:latest	0	0	4512
6	novelty	ollama_remote	glm4:latest	0	0	5801
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12190
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7563
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7704

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15022
11	judge	ollama_remote	glm4:latest	0	0	8493

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 100688 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/67d9d6a3fd41.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/67d9d6a3fd41.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/67d9d6a3fd41.tar.gz (if generated)